Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2
з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
“Розробка графічного редактора об’єктів”

Виконав:

Студент групи ІМ-55
Кузьменко Артур Олегович
Номер в списку - 11

Перевірив:

Київ 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту:

Індивідуальні параметри варіанту (для номера в журналі **Ж = 11**):

- **Структура масиву об'єктів:** Статичний масив вказівників Shape *pcshape[111].
- **Розмір масиву:** 111 елементів.
- **"Гумовий" слід при вводі:** Пунктирна лінія чорного кольору.
- **Прямокутник:**
    - *Спосіб вводу:* Від центру до одного з кутів.
    - *Відображення:* Чорний контур прямокутника з кольоровим заповненням.
    - *Колір заповнення:* Сірий.
- **Еліпс:**
    - *Спосіб вводу:* По двом протилежним кутам охоплюючого прямокутника.
    - *Відображення:* Чорний контур із кольоровим заповненням.
    - *Колір заповнення:* Білий.
- **Позначка поточного типу об'єкту:** У заголовку вікна.

Вихідний текст файлів:

main.cpp:

```
#include <windows.h>

#include <cmath>

#include "resource.h"

#include "shape.h"



#define MAX_SHAPES 111



HINSTANCE hInst;

LPCWSTR szTitle = L"Графічний редактор-[крапка]";

int currentShape = IDM_SHAPE_POINT;

bool isDrawing = false;

long xStart = 0, yStart = 0, xEnd = 0, yEnd = 0;

Shape* pcshape[MAX_SHAPES];

int shapeCount = 0;

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);

void DrawRubberBand(HWND hWnd, long x1, long y1, long x2, long y2, int
shapeType) {

    HDC hdc = GetDC(hWnd);

    int oldROP = SetROP2(hdc, R2_NOTXORPEN);

    HPEN hPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc,
GetStockObject(NULL_BRUSH));
```

```
    if (shapeType == IDM_SHAPE_POINT) {
        MoveToEx(hdc, x1, y1, NULL);
        LineTo(hdc, x1 + 1, y1 + 1);
    } else if (shapeType == IDM_SHAPE_LINE) {
        MoveToEx(hdc, x1, y1, NULL);
        LineTo(hdc, x2, y2);
    } else if (shapeType == IDM_SHAPE_RECT) {
        long dx = std::abs(x2 - x1);
        long dy = std::abs(y2 - y1);
        Rectangle(hdc, x1 - dx, y1 - dy, x1 + dx, y1 + dy);
    } else if (shapeType == IDM_SHAPE_ELLIPSE) {
        Ellipse(hdc, x1, y1, x2, y2);
    }
    SelectObject(hdc, hOldBrush);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
    SetROP2(hdc, oldROP);
    ReleaseDC(hWnd, hdc);
}
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR
lpCmdLine, int nCmdShow) {
    hInst = hInstance;
    WNDCLASSEXW wc = { 0 };
    wc.cbSize = sizeof(WNDCLASSEXW);
    wc.lpfnWndProc = WndProc;
```

```
    wc.hInstance = hInstance;

    wc.hCursor = LoadCursor(NULL, IDC_ARROW);

    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    wc.lpszMenuName = MAKEINTRESOURCEW(IDR_MAIN_MENU);

    wc.lpszClassName = L"Lab2WindowClass";

    if (!RegisterClassExW(&wc)) return 0;

    HWND hWnd = CreateWindowW(L"Lab2WindowClass", szTitle,
WS_OVERLAPPEDWINDOW,

        CW_USEDEFAULT, CW_USEDEFAULT, 800, 600, NULL, NULL, hInstance,
NULL);

    if (!hWnd) return 0;

    ShowWindow(hWnd, nCmdShow);

    UpdateWindow(hWnd);

    MSG msg;

    while (GetMessage(&msg, NULL, 0, 0)) {

        TranslateMessage(&msg);

        DispatchMessage(&msg);

    }

    return (int)msg.wParam;

}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam) {

    switch (message) {

    case WM_COMMAND:

        switch (LOWORD(wParam)) {

        case IDM_SHAPE_POINT:
```

```
            currentShape = IDM_SHAPE_POINT;
            SetWindowTextW(hWnd, L"Графічний редактор-[крапка]");
            break;
        case IDM_SHAPE_LINE:
            currentShape = IDM_SHAPE_LINE;
            SetWindowTextW(hWnd, L"Графічний редактор-[лінія]");
            break;
        case IDM_SHAPE_RECT:
            currentShape = IDM_SHAPE_RECT;
            SetWindowTextW(hWnd, L"Графічний редактор-[прямокутник]");
            break;
        case IDM_SHAPE_ELLIPSE:
            currentShape = IDM_SHAPE_ELLIPSE;
            SetWindowTextW(hWnd, L"Графічний редактор-[еліпс]");
            break;
        case IDM_FILE_EXIT:
            DestroyWindow(hWnd);
            break;
        }
        break;
    case WM_LBUTTONDOWN:
        if (shapeCount < MAX_SHAPES) {
            isDrawing = true;
            xStart = LOWORD(lParam);
            yStart = HIWORD(lParam);
```

```
            xEnd = xStart;
            yEnd = yStart;
            SetCapture(hWnd);
        }
        break;
    case WM_MOUSEMOVE:
        if (isDrawing) {
            DrawRubberBand(hWnd, xStart, yStart, xEnd, yEnd,
currentShape);
            xEnd = LOWORD(lParam);
            yEnd = HIWORD(lParam);
            DrawRubberBand(hWnd, xStart, yStart, xEnd, yEnd,
currentShape);
        }
        break;
    case WM_LBUTTONUP:
        if (isDrawing) {
            DrawRubberBand(hWnd, xStart, yStart, xEnd, yEnd,
currentShape);
            xEnd = LOWORD(lParam);
            yEnd = HIWORD(lParam);
            isDrawing = false;
            ReleaseCapture();
            if (shapeCount < MAX_SHAPES) {
                Shape* pShape = nullptr;
                if (currentShape == IDM_SHAPE_POINT) pShape = new
PointShape();
```

```
                else if (currentShape == IDM_SHAPE_LINE) pShape = new
LineShape();

                else if (currentShape == IDM_SHAPE_RECT) pShape = new
RectangleShape();

                else if (currentShape == IDM_SHAPE_ELLIPSE) pShape =
new EllipseShape();

                if (pShape) {

                    pShape->SetCoord(xStart, yStart, xEnd, yEnd);

                    pcshape[shapeCount++] = pShape;

                    InvalidateRect(hWnd, NULL, TRUE);

                }

            }

        }

        break;

    case WM_PAINT: {

        PAINTSTRUCT ps;

        HDC hdc = BeginPaint(hWnd, &ps);



        for (int i = 0; i < shapeCount; i++) {

            if (pcshape[i]) {

                pcshape[i]->Show(hdc);

            }

        }

        EndPaint(hWnd, &ps);

        break;

    }
```

```
    case WM_DESTROY:
        for (int i = 0; i < shapeCount; i++) {
            delete pcshape[i];
            pcshape[i] = nullptr;
        }
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProcW(hWnd, message, wParam, lParam);
    }
    return 0;
}
```

resource.h:

```
#pragma once


#define IDR_MAIN_MENU                   101
#define IDM_FILE_EXIT                   40001
#define IDM_SHAPE_POINT                 40002
#define IDM_SHAPE_LINE                  40003
#define IDM_SHAPE_RECT                  40004
#define IDM_SHAPE_ELLIPSE               40005
```

shape.cpp:

```cpp
#include "shape.h"

#include <cmath>



void Shape::SetCoord(long x1, long y1, long x2, long y2) {

    xs1 = x1;

    ys1 = y1;

    xs2 = x2;

    ys2 = y2;

}

//крапка

void PointShape::Show(HDC hdc) {

    SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));

}

//лінія

void LineShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);



    MoveToEx(hdc, xs1, ys1, NULL);

    LineTo(hdc, xs2, ys2);



    SelectObject(hdc, hOldPen);

    DeleteObject(hPen);

}

//прямокутник(11 вар:ввід від центру до кута, сіре заповнення)
```

```
void RectangleShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HBRUSH hBrush = CreateSolidBrush(RGB(128, 128, 128)); //сірий колір

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);



    long dx = std::abs(xs2 - xs1);

    long dy = std::abs(ys2 - ys1);

    Rectangle(hdc, xs1 - dx, ys1 - dy, xs1 + dx, ys1 + dy);



    SelectObject(hdc, hOldPen);

    SelectObject(hdc, hOldBrush);

    DeleteObject(hPen);

    DeleteObject(hBrush);

}

//еліпс(11 вар:ввід по 2 кутах, біле заповнення)

void EllipseShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HBRUSH hBrush = CreateSolidBrush(RGB(255, 255, 255)); //білий колір

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);



    Ellipse(hdc, xs1, ys1, xs2, ys2);



    SelectObject(hdc, hOldPen);
```

```
    SelectObject(hdc, hOldBrush);

    DeleteObject(hPen);

    DeleteObject(hBrush);

}
```

shape.h:

```
#pragma once

#include <windows.h>



class Shape {

protected:

    long xs1, ys1, xs2, ys2;

public:

    virtual ~Shape() {}

    void SetCoord(long x1, long y1, long x2, long y2);

    virtual void Show(HDC hdc) = 0;

};

class PointShape : public Shape {

public:

    void Show(HDC hdc) override;

};

class LineShape : public Shape {

public:

    void Show(HDC hdc) override;

};
```

```
class RectangleShape : public Shape {

public:

    void Show(HDC hdc) override;

};

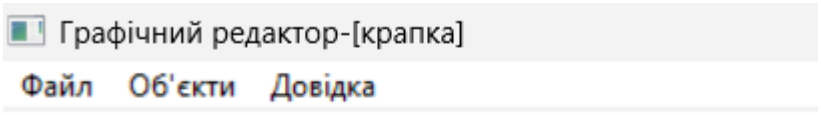
class EllipseShape : public Shape {

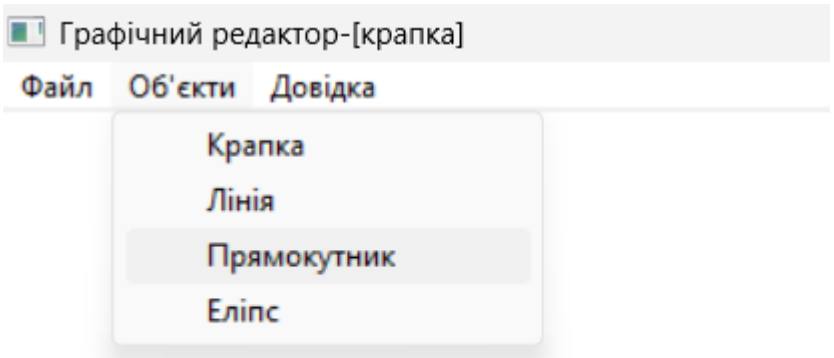
public:

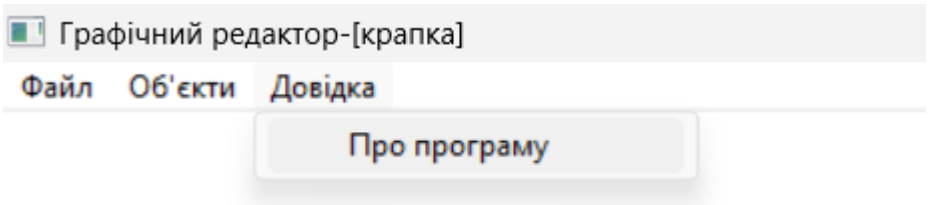
    void Show(HDC hdc) override;

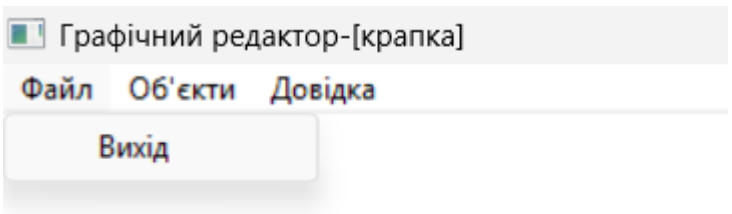
};
```

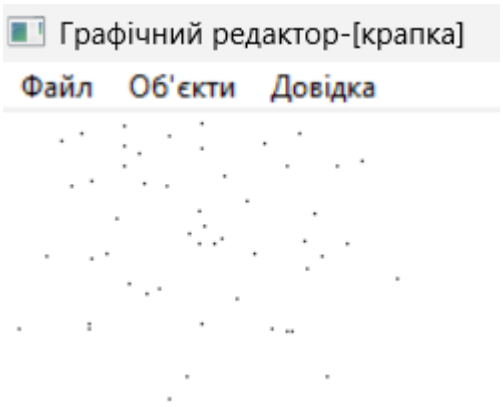
Скріншоти виконаної роботи:








Графічний редактор-[крапка]
Файл
Об'єкти
Довідка
Вихід


Графічний редактор-[крапка]
Файл
Об'єкти
Довідка


Графічний редактор-[лінія]
Файл
Об'єкти
Довідка

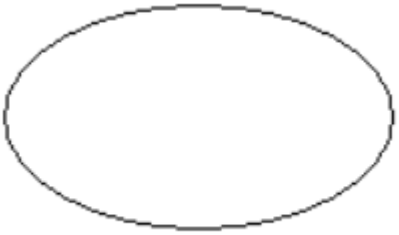
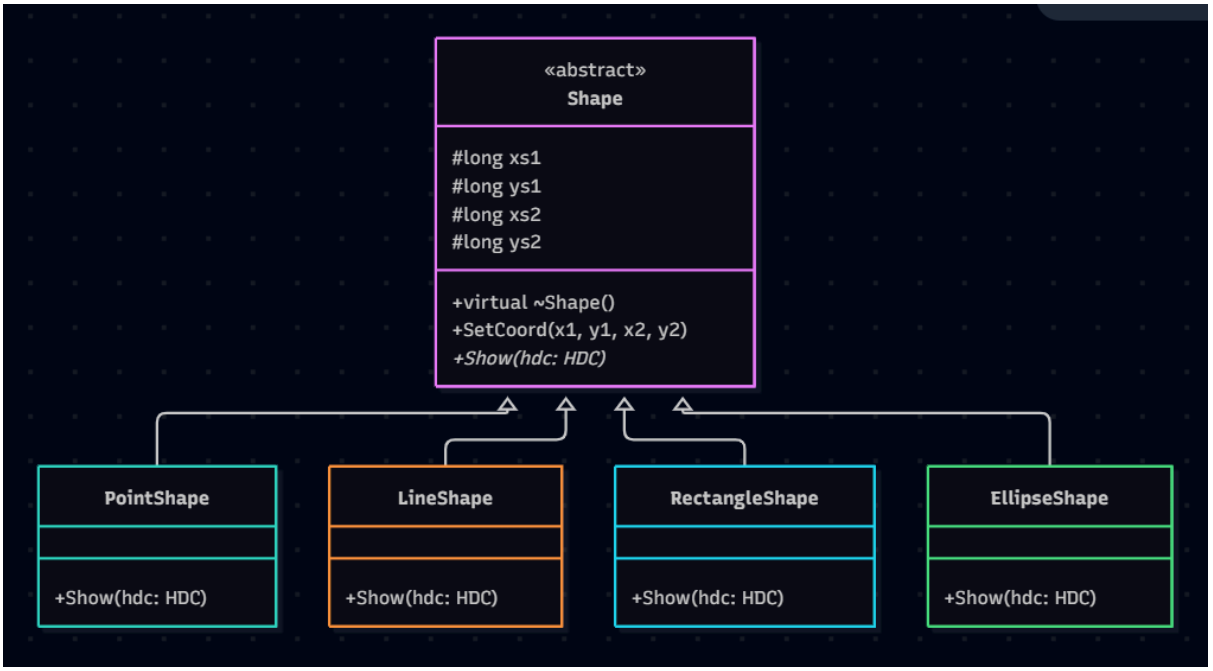




Схема успадкування класів:



Висновок: в ході виконання лабораторної роботи №2 я отримав вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши простий графічний редактор в об’єктно-орієнтованому стилі.